

개념 이해 가이드

그림으로 이해하는

RAG (검색 증강 생성)

LLM 이 거짓말을 줄이고 최신 정보를 답하게 만드는 원리
코드·수식 없이, 비유와 그림으로

대상: AWS Bedrock RAG 실습 수강생 (입문자)

작성 기준일: 2026-06-30

목차

1. 이 문서는.....	3
2. 왜 RAG 가 필요한가.....	3
3. RAG 란 무엇인가 — 4 단계 흐름	5
4. 핵심 개념 3 가지.....	6
5. 전체 그림으로 보기.....	8
6. RAG vs Fine-tuning.....	9
7. 자주 묻는 질문 (FAQ).....	10
8. 용어 한눈에 정리.....	11

1. 이 문서는

이 문서는 **RAG가 무엇이고 왜 필요한지**를 그림과 비유로 설명합니다. 코드나 수식은 전혀 없습니다. 실습(콘솔에서 지식봇 만들기)을 시작하기 전에 “**내가 지금 무엇을, 왜 만드는가**”를 분명히 잡는 것이 목표입니다.

순서는 이렇습니다. 먼저 **LLM의 한계**를 보고, 그것을 RAG가 어떻게 메우는지(2장), RAG의 **4단계 흐름**(3장), 그 안의 **핵심 개념 3가지**(4장), 마지막으로 **전체 그림**(5장)을 봅니다.

💡 한 문장 요약

RAG = LLM에게 답을 시키기 전에, 믿을 수 있는 문서를 먼저 찾아 손에 쥐여주는 것.

2. 왜 RAG가 필요한가

ChatGPT 같은 LLM은 똑똑하지만 두 가지 약점이 있습니다.

- **환각(Hallucination)** — 모르는 것도 그럴듯하게 지어냅니다.
- **지식 한계** — 학습한 시점 이후의 일이나, **우리 회사 내부 문서**는 알지 못합니다.

예를 들어 “우리 회사 데이터 엔지니어링 과목의 선수과목은?”이라고 물으면, 일반 LLM은 그 문서를 본 적이 없으므로 **아무 말이나 자신 있게** 답할 수 있습니다. 위험하죠.

RAG는 답하기 **전에** 관련 문서를 먼저 찾아 LLM에게 건네줍니다. LLM은 그 근거를 보고 답하므로, 지어내는 일이 크게 줄고 출처까지 제시할 수 있습니다.

일반 LLM vs RAG



▲ 일반 LLM은 기억에만 의존(환각 위험), RAG는 문서를 찾아 근거와 함께 답한다

비유 — 오픈북 시험

일반 LLM 은 아무 자료 없이 기억만으로 보는 시험 같습니다. RAG 는 오픈북 시험입니다. 답하기 전에 교과서에서 해당 페이지를 펴서 보고 답하니, 훨씬 정확하고 “몇 페이지에 있다”고 근거도 댈 수 있습니다.

3. RAG 란 무엇인가 — 4 단계 흐름

RAG 는 Retrieval-Augmented Generation(검색 증강 생성)의 약자입니다. 이름 그대로 **검색(Retrieval)**으로 **생성(Generation)**을 **강화(Augmented)**한다는 뜻입니다. 질문이 들어오면 다음 4 단계로 동작합니다.



질문이 들어오면 → 관련 문서를 찾아 → 질문에 붙여 → LLM이 근거를 보고 답한다

▲ RAG 의 4 단계: 질문 → 검색 → 증강 → 생성

단계	하는 일	쉽게 말하면
① 검색 (Retrieval)	질문과 의미가 비슷한 문서 조각을 벡터 DB 에서 찾음	질문에 맞는 페이지를 펼친다
② 증강 (Augmented)	찾은 문서 조각을 질문과 합쳐 프롬프트를 구성	질문 옆에 근거 자료를 붙인다
③ 생성 (Generation)	LLM 이 그 근거를 보고 답변을 작성	자료를 보고 답을 쓴다
출처 표시	답변이 어떤 문서에서 나왔는지 함께 제시	“○○ 문서 참고”라고 밝힌다

핵심은 **LLM 을 다시 학습시키지 않는다**는 점입니다. 문서만 갈아 끼우면 답도 바뀝니다. 그래서 빠르고 저렴합니다.

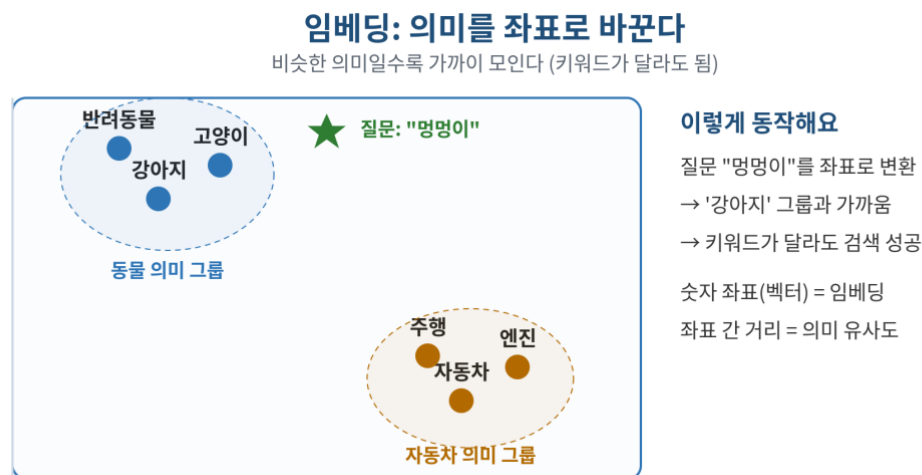
4. 핵심 개념 3 가지

RAG 를 이해하려면 세 단어만 알면 됩니다: 임베딩, 벡터 데이터베이스, 청킹.

4.1 임베딩 — 의미를 좌표로 바꾸기

컴퓨터는 글의 뜻을 직접 이해하지 못합니다. 그래서 문장을 숫자 좌표(벡터)로 바꾸는데, 이 변환을 임베딩이라고 합니다. 신기한 점은 뜻이 비슷한 말은 좌표도 가깝게 찍힌다는 것입니다.

그래서 “멍멍이”라고 물어도 “강아지”가 적힌 문서를 찾아냅니다. 단어가 똑같지 않아도, 의미가 가까우면 검색됩니다. 키워드 검색과의 결정적 차이죠.



4.2 벡터 데이터베이스 — 좌표를 저장하고 빠르게 찾기

문서 조각들을 임베딩한 좌표(벡터)를 저장하는 특수한 창고가 벡터 데이터베이스입니다. 질문이 들어오면 질문 좌표와 가장 가까운 문서 조각들을 순식간에 찾아줍니다. 우리 실습에서는 Bedrock 이 이 창고(OpenSearch 또는 S3 Vectors)를 자동으로 만들어 줍니다.

💡 키워드 검색 vs 의미 검색

기존 검색은 단어가 정확히 일치해야 찾습니다(“멍멍이”로는 “강아지” 문서를 못 찾음). 벡터 검색은 의미가 비슷하면 찾으므로 오타·동의어·다른 표현에도 강합니다.

4.3 청킹 — 긴 문서를 조각으로 나누기

문서를 통째로 넣으면 검색이 부정확하고 LLM 입력 한도를 넘깁니다. 그래서 문서를 **의미 단위의 작은 조각(청크)**으로 자릅니다. 조각끼리 **약간 겹치게(overlap)** 잘라, 문장이 중간에 끊겨 문맥이 사라지는 것을 막습니다.



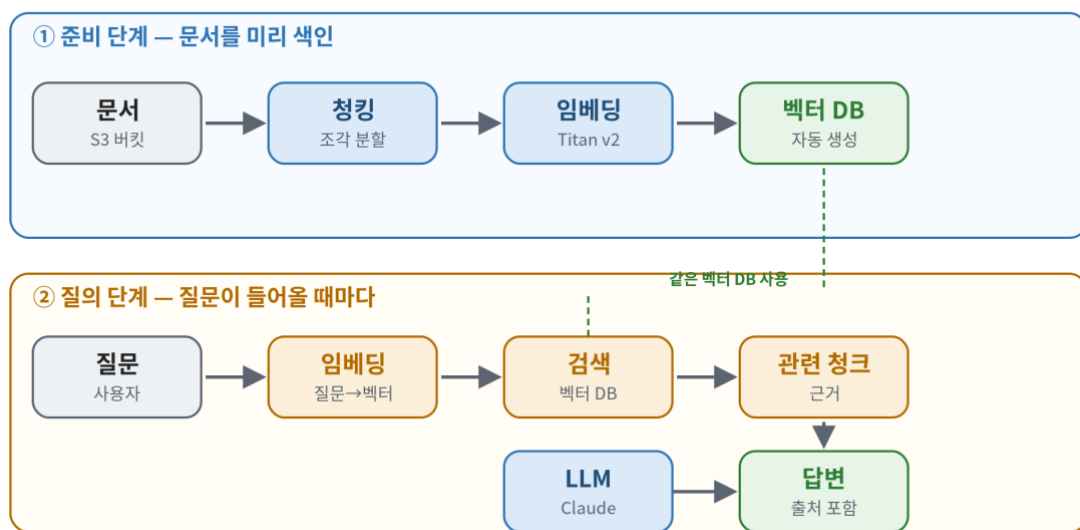
▲ 긴 문서를 일부 겹치게 조각내면 검색 정확도가 올라간다

5. 전체 그림으로 보기

지금까지의 개념을 하나로 합치면 RAG 는 **두 단계**로 나뉩니다.

- ① **준비 단계(색인)** — 문서를 미리 청킹·임베딩해 벡터 DB 에 저장. 처음 한 번, 그리고 문서가 바뀔 때만.
- ② **질의 단계** — 질문이 들어올 때마다 질문을 임베딩→검색→근거와 함께 LLM 이 답변.

RAG 전체 구조 (Amazon Bedrock 기준)



▲ 준비 단계(문서 색인)와 질의 단계(질문 응답)는 같은 벡터 DB 를 공유한다

💡 이 그림이 곧 우리 실습

실습에서 S3 에 문서 업로드 = 준비 단계의 시작, Knowledge Base 생성·Sync = 청킹·임베딩·벡터 DB 저장, 콘솔 Test 질의 = 질의 단계 입니다. 개념과 실습이 1:1 로 연결됩니다.

6. RAG vs Fine-tuning

“모델을 우리 데이터로 다시 학습(Fine-tuning)시키면 되지 않나?”라는 질문이 많습니다. 둘은 목적이 다릅니다.

항목	RAG	Fine-tuning
방식	답할 때 문서를 찾아 참고	모델 자체를 재학습
데이터 최신성	문서만 바꾸면 즉시 반영	바뀔 때마다 재학습 필요
비용	검색 비용 위주, 저렴	대규모 GPU 학습 비용
환각	출처 기반이라 낮음	여전히 발생 가능
잘 맞는 일	사내 문서 Q&A, 지식 검색	특정 말투·문체·도메인 표현 학습
난이도	상대적으로 쉬움	높음

정리하면, “무엇을 아는가”를 바꾸려면 RAG, “어떻게 말하는가”를 바꾸려면 Fine-tuning 이 적합합니다. 사내 지식봇은 거의 항상 RAG 입니다. 둘을 함께 쓰기도 합니다.

7. 자주 묻는 질문 (FAQ)

Q. 문서에 없는 걸 물으면 어떻게 되나요?

잘 만든 RAG 는 “자료에 없다”고 답하도록 유도됩니다. 실습에서 “요리 레시피 추천”처럼 문서에 없는 질문을 던져, 봇이 모른다고 하는지 확인해 보세요. 그것이 환각이 억제된 정상 동작입니다.

Q. 그래도 가끔 틀린 답을 합니다.

검색이 엉뚱한 조각을 가져왔거나, 근거가 부족한데도 LLM 이 답을 지어낸 경우입니다. 검색 결과 개수(청크 수)를 늘리거나, 청킹 방식을 바꾸거나, 질문을 더 구체적으로 하면 개선됩니다.

Q. 문서가 자주 바뀌면요?

RAG 의 가장 큰 장점입니다. S3 의 문서를 갈아 끼우고 **Sync** 만 다시 누르면 끝입니다. 모델 재학습이 전혀 필요 없습니다.

Q. Azure 에서 해봤는데 비슷한가요?

네. **Azure OpenAI On Your Data** 와 거의 같은 개념입니다. **Bedrock** $\approx$ **Azure OpenAI**, **OpenSearch/S3 Vectors** $\approx$ **Azure AI Search** 로 대응시키면 됩니다.

8. 용어 한눈에 정리

용어	뜻
RAG	답하기 전에 문서를 검색해 근거로 삼는 기법
LLM	대규모 언어 모델 (예: Claude, GPT)
환각(Hallucination)	모델이 사실이 아닌 내용을 그럴듯하게 지어내는 현상
임베딩(Embedding)	문장을 의미가 담긴 숫자 좌표(벡터)로 바꾸는 것
벡터 DB	임베딩 벡터를 저장하고 유사한 것을 빠르게 찾는 데이터베이스
청킹(Chunking)	긴 문서를 작은 조각으로 나누는 것
Knowledge Base	문서·임베딩·벡터 DB 를 묶은 Bedrock 의 RAG 구성 단위
Sync(동기화)	S3 문서를 청킹·임베딩해 벡터 DB 에 반영하는 작업
Titan Embeddings	AWS 의 임베딩 모델 (문서를 벡터로 변환)
Claude	답변을 생성하는 LLM (Anthropic)

다음 단계

개념이 잡혔다면, 실습 가이드 「**Amazon Bedrock 으로 만드는 RAG 사내 지식봇**」으로 넘어가세요. 이 문서의 그림들이 실제 화면에서 어떻게 구현되는지 직접 만들어 봅니다.